

Dönüştür ve Yönet

(devam)

Find the missing number in an array.

Input:- 1, 2, 4, 6, 3, 7, 8

Output:- 5

```
//find missing number in array of size
int findMissingNumberSort(int arr[], int size)
{
    int i;

    //sort the array
    quickSort(arr,size);

    //loop through integers 1 to size
    for(i = 0;i<size;i++)
    {
        //if the index does not match, return
        if((i+1) != arr[i])
            return i;
    }
}
```

Time Complexity:- $O(n\log(n))$ based on the sorting technique.

Space Complexity:- $O(1)$

Find the missing number in an array.

Input:- 1, 2, 4, 6, 3, 7, 8

Output:- 5

```
//find missing number in array of size 'size'
int findMissingNumberSummation(int arr[], int size)
{
    int i;

    int sum = ((size) * (size+1))/2;

    int arr_sum = 0;

    //loop through the array and calculate array sum
    for(i = 0; i < size; i++)
    {
        arr_sum += arr[i];
    }

    return (sum - arr_sum);
}
```

Time Complexity:- $O(n)$ for scanning the complete array

Space Complexity:- $O(1)$

Find the missing number in an array.

$$A \oplus A = 0.$$

Time Complexity:- $O(n)$ to scan the array

Space Complexity:- $O(1)$

Input:- 1, 2, 4, 6, 3, 7, 8

Output:- 5

```
//find missing number in array of size 'size' range is 1-n
int findMissingNumberXOR(int arr[], int size, int n)
{
    int i;

    int xor1 = 0;
    //get the XOR of all the numbers from 1 to n
    for(i=1;i<=n;i++)
        xor1 ^= i;

    int xor2 = 0;

    //loop through the array and get the XOR of elements
    for(i = 0;i<size;i++)
    {
        xor2 ^= arr[i];
    }

    return (xor1 ^ xor2);
}
```

- 1'den n'e kadar olan tüm sayıları XOR'la.
- 'num1' değerini al.
- Dizinin tüm elemanlarını XOR'la.
- 'num2' değerini al.
- num1 ve num2'nin XOR'unu al.

Tüm elemanlar tekrarlandığından, XOR 0 olur ve sıfır olmayan tek sayı eksik sayıdır. Böylece kayıp sayımızı elde ederiz.

Problem: Kayıp tam sayı bulma: Bir dizi $A[1, \dots, n]$, 0 ile n arasındaki tüm tam sayıları içeriyor, ancak bu sayılardan biri eksik. Yardımcı bir dizi $B[0, \dots, n]$ kullanarak A 'da hangi sayıların görüldüğünü işaretleyerek eksik olan sayıyı $O(n)$ zamanda bulmak, $\Theta(n)$ algoritmalar incelenmişti. Ancak bu soruda, A dizisindeki bir tam sayıya doğrudan erişim izniniz yok. A 'nın elemanları ikili (binary) biçimde temsil ediliyor ve yalnızca şu işlemi kullanabiliyoruz: “ $A[i]$ 'nin j 'inci bitini getir”, bu işlem sabit zamanda ($O(1)$) gerçekleştirilebiliyor.

Çözüm:

n sayısını ikili olarak temsil etmek için gereken bit sayısına m diyelim. Bilindiği üzere, $m = \lfloor \log_2 n \rfloor + 1$ olur. Bu problemde, A dizisini bir $m \times n$ boyutunda, yalnızca 0 ve 1'lerden oluşan bir matris gibi düşüneceğiz. A 'nın m numaralı satırı, dizideki sayıların en az anlamlı (least significant) bitlerini içerir; $m-1$ numaralı satır, ikinci en az anlamlı bitleri içerir; bu şekilde devam eder ve 1 numaralı satır, en anlamlı (most significant) bitleri içerir.

Örneğin, $n = 10$ ve eksik sayı $6 = 0110_2$ (ikilik tabanda 6) ise, A matrisi şu şekilde görünebilir:

$A =$

0	0	1	0	0	0	0	1	1	0
0	0	0	1	1	1	0	0	0	0
1	0	1	1	0	0	0	0	0	1
1	1	0	1	0	1	0	1	0	0

Tam bir dizi (complete array), boyutu n olan ve yukarıda tanımlanan **A**'ya benzeyen, fakat içinde eksik bir sütun (column) bulunmayan iki boyutlu bir bit dizisi olarak tanımlanır. Açıktır ki, böyle bir dizide toplam $n + 1$ **sütun** bulunur.

Örneğin, boyutu **10** olan tam bir dizi şu şekilde olurdu:

0	0	1	0	0	0	0	1	0	1	0
0	0	0	1	1	1	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	1
1	1	0	1	0	1	0	1	0	0	0

Boyutu n olan eksik bir dizi (almost complete array), tam olarak bir sütunu eksik olan iki boyutlu bir bit dizisi olarak tanımlanır.

Şimdi boyutu n olan herhangi bir **tam dizi**yi, yani $\tilde{\mathbf{A}}$ 'yı ele alalım.

$\tilde{\mathbf{L}}$, bu dizinin en alt satırını (bottom row) temsil etsin.

Bu satır, $\tilde{\mathbf{A}}$ 'daki sayıların en az anlamlı bitlerinden oluşur.

Şimdi şu şekilde tanımlayalım:

- $\tilde{n}_0$, $\tilde{\mathbf{L}}$ satırındaki **sıfır (0)** sayısıdır.
- $\tilde{n}_1$, $\tilde{\mathbf{L}}$ satırındaki **bir (1)** sayısıdır.
- $\tilde{\Delta} = \tilde{n}_0 - \tilde{n}_1$ şeklinde tanımlansın.

$$\tilde{\Delta} = \begin{cases} 0 & \text{eğer } n \text{ tek ise} \\ 1 & \text{eğer } n \text{ çift ise} \end{cases}$$

Gerçekten de, basit bir tümevarım (indüksiyon) ile şu ispatlanabilir:
Eğer $\tilde{\mathbf{A}}$ dizisindeki sütun sayısı $(n + 1)$ **çift** ise, o zaman $\tilde{\Delta} = 0$ olur;
eğer **tek** ise, o zaman $\tilde{\Delta} = 1$ olur.

Şimdi **A** dizisini ele alalım: Bu, $\sim A$ 'dan bir sütunun (yani eksik sayının) silinmesiyle elde edilen, boyutu **n** olan herhangi bir eksik elemanlı (almost complete array)dir.

L, **A** dizisinin en alt satırı (bottom row) olsun. Bu satır, dizideki sayıların en az anlamlı bitlerini içerir.

İddiamız şudur:

- n_0 , **L**'deki **0** sayısı,
- n_1 , **L**'deki **1** sayısı,
- $\Delta = n_0 - n_1$ şeklinde tanımlansın.

$$\Delta = \begin{cases} \bar{\Delta} + 1, & \text{eğer eksik sayı tek ise} \\ \bar{\Delta} - 1, & \text{eğer eksik sayı çift ise} \end{cases}$$

Gerçekten de, eğer eksik olan sayı **çift** ise, **L** satırında $\sim L$ 'ye kıyasla bir tane daha az **0** bulunur, fakat **1** sayısı aynıdır; yani: $n_0 = \sim n_0 - 1$ ve $n_1 = \sim n_1$.

Benzer şekilde, eğer eksik olan sayı **tek** ise, **L**'de $\sim L$ 'ye kıyasla bir tane daha az **1** bulunur, fakat **0** sayısı aynıdır; yani: $n_0 = \sim n_0$ ve $n_1 = \sim n_1 - 1$.

$$\Delta = \begin{cases} 2, & \text{Eğer n ÇİFT ve eksik Sayı TEK ise} \\ 1, & \text{Eğer n TEK ve eksik Sayı TEK ise} \\ 0, & \text{Eğer n ÇİFT ve eksik Sayı ÇİFT ise} \\ -1, & \text{Eğer n TEK ve eksik Sayı ÇİFT ise} \end{cases}$$

Şu sonuca varıyoruz ki:

- $\Delta \in \{1, 2\}$ ise $\rightarrow$ eksik sayının en az anlamlı biti **1**'dir.
- $\Delta \in \{-1, 0\}$ ise $\rightarrow$ eksik sayının en az anlamlı biti **0**'dır.

Yani, **A dizisinin en alt satırı boyunca yapılacak tek bir lineer tarama ile Δ değeri hesaplanabilir** ve ardından sabit sürede eksik sayının en az anlamlı biti bulunabilir.

$$n + \frac{n}{2} + \frac{n}{4} + \dots + 1 \leq 2n = \Theta(n)$$

Ancak, aynı yaklaşımı eksik sayının diğer bitleri için doğrudan kullanırsak, her seferinde dizinin tüm sütunlarını taramak gerekeceğinden işlem süresi **$O(n \log n)$** olur. Çünkü satır sayısı **n** 'in logaritması kadardır.

Buradaki **kilit gözlem** şudur:

Eksik sayının **ikinci en az anlamlı bitini** bulmak için tüm sütunları değil, sadece yaklaşık **yarısını** taramamız yeterlidir.

- Eğer en az anlamlı bit **1** olarak bulunmuşsa, ikinci bitin hesaplanmasında **en alt satırında 1 olan sütunlar** dikkate alınır.
- Eğer en az anlamlı bit **0** olarak bulunmuşsa, ikinci bitin hesaplanmasında **en alt satırında 0 olan sütunlar** dikkate alınır.

```

int m = floor(logb(n)) + 1;
int A[m][n];

int main() {
    int i, j, t, numrow, n0, n1, ntemp = n;
    int B[n], res[m];
    for(i = 0; i < n; i ++)
        B[i] = i;
    for(i = m - 1; i >= 0; i --) {

        n0 = n1 = 0;
        for(j = 0; j < ntemp; j ++) {
            if (A[i][B[j]] == 0) n0 ++;
            else n1 ++; }
        if(n0 - n1 == 2 || n0 - n1 == 1) res[i] = 1;
        if(n0 - n1 == 0 || n0 - n1 == -1) res[i] = 0;
        for(j = 0, t = 0; j < ntemp; j ++)
            if((res[i] == A[i][B[j]])) {
                B[t] = B[j];
                t ++; }
        if(ntemp % 2 == 0) ntemp = (ntemp / 2);
        else if(res[i] == 0) ntemp = floor(ntemp / 2);
        else ntemp = ceil(ntemp / 2);
    }
    for(i = 0; i < n; i ++)
        printf("%d", res[i]);
}

```

Bu dizide sabit zamanda erişilebilecek tek biçim $A[j][i]$ şeklindedir.

Bu erişim biçimi “ **$A[i]$** ’nin **j** ’inci bitini getir” şeklinde adlandırılır.

Burada **ilk bit en anlamlı bit (most significant bit)** olarak kabul edilir, sonraki bitler azalan öneme sahiptir.

Aşağıda C dilinde yazılmış bir programı göz önünde bulunduralım:

Şimdi neden bunun doğru olduğunu açıklayalım: Dizinin satır sayısı $m = \lfloor \log_2 n \rfloor + 1$ 'dir.

Şöyle tanımlayalım:

- $A_0^{(m-1)}$, A dizisinden **(m-1). satırda 0 bulunan sütunların silinmesi**, ardından **(m-1). satırın silinmesiyle** elde edilen iki boyutlu dizidir.
- $A_1^{(m-1)}$, A dizisinden **(m-1). satırda 1 bulunan sütunların silinmesi**, ardından **(m-1). satırın silinmesiyle** elde edilen dizidir.

$$A =$$

0	0	1	0	0	0	0	1	1	0
0	0	0	1	1	1	0	0	0	0
1	0	1	1	0	0	0	0	0	1
1	1	0	1	0	1	0	1	0	0

$$A_0^{(3)} =$$

0	0	0	0	1
0	0	1	1	0
1	0	1	0	0

$$A_1^{(3)} =$$

1	0	0	1	0
0	1	0	0	0
1	0	0	0	1

Aynı işlemler bir eleman kalıncaya kadar devam ettirilir. Derste tamamı çözüldü.

Problem : A dizisindeki eksik sayıyı bulma probleminde, **dizideki sayıların sıralı olması**(bit düzeninde gösterilmişti ve sütunların da sıralı düzende verildiğini bilerek) durumunda algoritmanın çalışma zamanı maliyetini hesaplayınız. A'ya $O(\lg n)$ bit erişimi mümkün müdür?

Çözüm: A'daki sayılar sıralanırsa, sorun önce eksik sayının en anlamlı biti, sonra ikinci en anlamlı biti, vb. yani en az anlamlı biti belirlenerek çözülebilir; böylece her bit için A'ya yalnızca bir erişim hakkı elde edilir.

Diyelim ki $\tilde{A}$, n boyutundaki tam dizidir, yani eksik sayı yoktur. Örneğin, $n = 10$

$\tilde{A} =$

0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	1	1	1	1	0	0	0
0	0	1	1	0	0	1	1	0	0	1
0	1	0	1	0	1	0	1	0	1	0
0	1	2	3	4	5	6	7	8	9	10

Sıfırlar ve birler arasındaki en üst satırdaki sınırı düşünün. En sağdaki sıfır 7. sütundadır ($7 = 2^{\log_2 10} - 1$) ve en soldaki sıfır 8. sütundadır ($8 = 2^{\log_2 10}$). En soldaki sütun 0 sayısı olduğu sürece, sınırın $2^{\log_2 n} - 1$ ve $2^{\log_2 n}$ sütunları arasında olduğunu genelleştirmek mümkündür.

Şimdi, n boyutundaki neredeyse tamamlanmış bir A dizisindeki sıfırlar ve birler arasındaki en üst satırdaki sınırı düşünün. Örneğin, $n = 10$ ve eksik sayı $6 = 0110_b$ ise, A şudur:

$A =$

0	0	0	0	0	0	0	1	1	1
0	0	0	0	1	1	1	0	0	0
0	0	1	1	0	0	1	0	0	1
0	1	0	1	0	1	1	0	1	0
0	1	2	3	4	5	6	7	8	9

Üst sıradaki $7 = 2^{\log_2 10} - 1$ ve $8 = 2^{\log_2 10}$ pozisyonlarını ele alalım. Şimdi sınır aralarında değildir çünkü eksik sayının en önemli biti 0'dır, bu yüzden sınır $\sim A$ ile karşılaştırıldığında bir pozisyon sola "kaydırılır". Ancak, eksik sayının en önemli bitinin ne olduğunu bilmiyorsak, üst sıradaki 7 ve 8 pozisyonlarında iki 1 olması gerçeğinden bunun 0 olduğunu çıkarabiliriz.

Açıkça, eğer en üst satırda sırasıyla 7 ve 8 pozisyonlarında 0 ve 1 olsaydı, bu eksik sayının en önemli bitinin (msb) 1 olacağı anlamına gelirdi, tıpkı dizinin B , $n = 10$ ve eksik sayının $9 = 1001_b$ olduğu yandaki örnekte olduğu gibi:

$B =$

0	0	0	0	0	0	0	0	1	1
0	0	0	0	1	1	1	1	0	0
0	0	1	1	0	0	1	1	0	1
0	1	0	1	0	1	0	1	0	0
0	1	2	3	4	5	6	7	8	9

Yani, sadece en üst satırdaki $2^{\log_2 n} - 1$ pozisyonunu inceleyerek, eksik sayının en önemli bitini aşağıdaki gibi çıkarabiliriz:

$$\text{eksik sayının en önemli biti} = \begin{cases} 0, & \text{if } A[0][2^{\lfloor \log_2 n \rfloor} - 1] = 1 \\ 1, & \text{if } A[0][2^{\lfloor \log_2 n \rfloor} - 1] = 0 \end{cases}$$

Maliyet: Eksik sayının en önemli bitini $\Theta(1)$ süresinde hesapladıktan sonra, ikinci en önemli bit $\Theta(1)$ süresinde hesaplarız, vb., böylece eksik sayının tüm bitlerini $\Theta(\lg n)$ denemeyle hesaplayabiliriz.

Durum I: En önemli bit 0 ise, ikinci en önemli biti hesaplamak için yalnızca $0, 1, \dots, 2^{\log_2 n} - 2$ sütunları ile $1, 2, \dots, \log_2 n$ satırlarından oluşan alt diziyi dikkate alırız.

A'yı bir örnek olarak kullanırsak, bu alt dizi A' aşağıdaki gibi olur:

A =	0	0	0	0	0	0	0	1	1	1
	0	0	0	0	1	1	1	0	0	0
	0	0	1	1	0	0	1	0	0	1
	0	1	0	1	0	1	1	0	1	0
	0	1	2	3	4	5	6	7	8	9

$\mapsto$

A' =	0	0	0	0	1	1	1
	0	0	1	1	0	0	1
	0	1	0	1	0	1	1
	0	1	2	3	4	5	6

A' ile tıpkı A ile olduğu gibi **yinelemeli** olarak ilerliyoruz çünkü A' eksik bir matris. A'nın boyutunu hesaplayalım. Başlangıçtaki A herhangi bir boyutta olabilir ancak A'nın boyutu n tarafından benzersiz bir şekilde belirlenir. $m = \log_2 n + 1$ olsun, yani n'yi ikili sistemde temsil etmek için gereken bit sayısı (bu durumda m, A' matrisindeki daki satır sayısıdır). $m' = m - 1$ olsun. Türetilen A' matrisinin boyutu $2^{m'} - 1$ 'dir. Buradaki “-1”, A' da eksik bir sayı olmasından kaynaklanmaktadır.

Tamamı derste çözüldü.

Durum II: **Eğer en önemli bit 1** ise, ikinci en önemli biti hesaplamak için yalnızca en üst satırda sütunlarında 1'ler bulunan alt diziyi dikkate alırız. Yukarıdaki B'yi bir örnek olarak kullanarak, bu **B'** alt matrisini aşağıdaki gibi elde ederiz:

B =	0	0	0	0	0	0	0	1	1
	0	0	0	0	1	1	1	0	0
	0	0	1	1	0	0	1	0	1
	0	1	0	1	0	1	0	0	0
	0	1	2	3	4	5	6	7	8

→

B' =	0	0
	0	1
	0	0
	0	1

B' matrisindeki sütun sayısını (bunu **c'** olarak adlandıralım) hesaplamak kolaydır:

$c' = n - 2^{m'}$ olarak hesaplanır. Burada $m' = \log_2 n$ dir. Somut örnekte, $n = 10$, $m' = 3$, dolayısıyla $c' = n - 2^{m'} = 10 - 8 = 2$.

Bu işlem, B0'dan gerekli sayıda üst satırı kaldırmaya eşdeğerdir. İşlem sona erdiğinde ve B0 gerektiği gibi azaltıldığında, yinelemeli olarak ele alınabilir.

Ancak, **B'** mutlaka neredeyse tamamlanmış bir dizi değildir; neredeyse tamamlanmış bir dizi olabilmesi için en üst satırında en az bir 1 olması gerekir. Aslında, yalnızca $c' > 1/2 (2^{m'}) = 2^{m'-1}$ olduğunda “neredeyse tamamlanmış” bir dizidir. Bu örnekte **B'** eksik bir dizi değildir çünkü fazla bir satırı var. Eksik sayının ikinci en önemli bitinin 0 olduğunu (eksik sayı $9 = 1001_b$) sadece **B'** matrisinin boyutlarını bilerek çıkarabiliriz; bu sonuca varmak için en üst sıranın bitlerini taramamıza veya incelememize gerek yok. Kural, $c' \leq 2^{m'-1}$ olduğu sürece, eksik sayıların bit pozisyonlarına 0'ları yazıp $m' = m' - 1$ işlemini gerçekleştirmektir.

Tamamı derste çözüldü

```

int m = floor(logb(n)) + 1;
int A[m][n], res[m];
void find(int, int, int) ;

int main() {
    find(0, n-1, 0);
    return 0; }

void find(int low, int high, int row) {
    int j, c, n1, n2, numel = high - low + 1;
    if(row == m-1) {
        res[row] = !(A[i][0]);
        PrintResult();
        return; }
    n1 = floor(logb(numel));
    n2 = 1 << n1;
    if(A[row][low+n2-1] == 1) {
        res[row] = 0;
        find(low, n2-2, row+1); }
    else {
        j = 1;
        res[row] = 1;
        c = numel - n2;
        while((n1 >= 0) && c <= (1<<(--n1))) {
            j ++;
            res[row+j] = 0; }
        if(row+j == m-1) {
            PrintResult();
            return; }
        find(n2, high, row+j); } }

```

Zaman karmaşıklığı $\Theta(\lg n)$ 'dir

TEŞEKKÜRLER